

CocktailMachines B.V.

Invoice Automation System

n8n Workflow Suite — Complete Technical Documentation

6	40+	3	2
Workflows	Nodes	AI Assistants	Databases

Project	CocktailMachines Invoice Processing Automation
Version	1.0 — Production
Platform	n8n Workflow Automation
AI Models	OpenAI GPT-4.1 + OpenAI Assistants API
Database	MySQL (Test) + MySQL (Production)
Integrations	Gmail, FTP, PDF.co API, OpenAI Vector Store
Date	March 2025

TABLE OF CONTENTS

1	Project Overview & Architecture	3
2	System Integration Map	4
3	Main Flow — Email Ingestion & OCR Pipeline	5
4	A1 — Resolve Invoice Supplier	9
5	A2 — Invoice Classifier & Transformer	12
6	A3 — Purchase Order Matcher	16
7	A4 — Ledger Mapper	19
8	Vector Store Updater — Supplier Knowledge Base	22
9	Database Schema & Tables	25
10	Error Handling & Fallback Strategy	27

1. Project Overview & Architecture

About CocktailMachines Invoice Automation

CocktailMachines B.V. operates a fully automated invoice processing pipeline built on n8n, the open-source workflow automation platform. This system eliminates manual invoice handling by automatically ingesting invoices via Gmail, extracting text through OCR, classifying and validating invoice data using OpenAI's Assistant API, matching invoices to purchase orders, and updating the financial ledger — all without human intervention.

Core Objectives

Zero-touch Invoice Processing	Invoices arrive via email and are processed end-to-end automatically, from OCR extraction to database insertion.
AI-Powered Classification	OpenAI GPT-4.1 Assistants classify each invoice as 'inkoopfactuur' (purchase), 'kostenfactuur' (cost), or 'onbekend' (unknown).
Supplier Resolution	An AI assistant cross-references invoice text against a vector-store knowledge base of known suppliers to identify the correct creditor.
Purchase Order Matching	For purchase invoices, the system automatically matches invoice line items against open purchase orders from the production ERP database.
Ledger Mapping	Cost invoices are automatically assigned the correct general ledger codes via an AI Ledger Mapper assistant.
Audit Trail	Every step is logged in MySQL with status updates, ensuring complete traceability.

Workflow Architecture Overview

The system is composed of one Main Flow and five Sub-Workflows (A1–A4 + Vector Store Updater). The Main Flow orchestrates the entire process, calling sub-workflows as needed. Each sub-workflow is a specialised, independently executable unit.

Workflow	Role	Trigger	AI?
Main Flow	Orchestrator: email → OCR → routing	Gmail (every minute)	Yes (GPT-4.1 cleanup)
A1 — Supplier Resolver	Identify supplier from invoice text	Sub-workflow call	Yes (OpenAI Assistant)
A2 — Classifier	Classify invoice type + transform to UBL	Sub-workflow call	Yes (OpenAI Assistant)
A3 — PO Matcher	Match invoice to open purchase orders	Sub-workflow call	Yes (OpenAI Assistant)
A4 — Ledger Mapper	Map cost invoice to GL accounts	Sub-workflow call	Yes (OpenAI Assistant)
Vector Store Updater	Keep supplier AI knowledge base fresh	Manual trigger	Yes (GPT-4.1)

2. System Integration Map

External Services & Credentials

Service	Purpose	Credential Used	Flow(s)
Gmail (OAuth2)	Receive invoices as email attachments	Gmail Noor	Main Flow
MySQL Test DB	Store email logs, processed files, invoice JSONs	MySQL account Test	Main, A1, A2
MySQL Prod DB	Read open purchase orders (ERP)	MySQL Prod account	Main, Vector Updater
PDF.co API	Upload PDF + convert to plain text (OCR)	API Key (info@cocktailmachines)	Main Flow
OpenAI Assistants	A1 Supplier Resolver, A2 Classifier, A3 PO Mapping, A4 Invoice Mapper	OpenAI Account test	A1, A2, A3, A4
OpenAI GPT-4.1	Reformat OCR text + Supplier profile enrichment	OpenAI account test	Main, Vec. Updater
OpenAI Vector Store	Store supplier knowledge chunks for AI retrieval	val_6826ef5e64a0819182f331d03f6527e8	Vec. Updater
FTP Server (Prod)	Store invoice PDFs (errors/ and new/ folders)	FTP Prod account	Main Flow
Mistral Cloud	Alternative OCR text extraction (image invoices)	Mistral Cloud account	Main Flow

File Path Conventions (FTP)

FTP Path	Content	When Written
facturen/errors/	All received invoice files (initial backup)	On every new invoice arrival
facturen/new/	Successfully processed invoices	After successful AI processing

3. Main Flow — Email Ingestion & OCR Pipeline

Flow Overview

The Main Flow is the central orchestrator of the entire invoice processing system. It polls a dedicated Gmail inbox every minute, downloads invoice attachments (PDF, JPEG, or XML), processes each file through OCR or XML parsing, resolves the supplier, and routes the classified invoice data to the appropriate downstream sub-workflow (A1 through A4). It also handles FTP file management, database logging, and PDF credit tracking.

Trigger Gmail OAuth2 — polls every minute for new emails with attachments (PDF, JPEG, or XML). Downloads attachments as binary data.

Input Email with one or more invoice attachments (PDF, image, or UBL XML)

Output Fully processed invoice stored in DB, FTP, and routed to sub-workflows for matching/ledger

Phase 1: Email Ingestion & File Parsing

#	Node Name	Type	Description
1	Gmail Trigger1	Gmail Trigger	Polls Gmail (Noor account) every minute. Downloads all email attachments. Passes raw binary data downstream.
2	Wait 20s	Wait	Waits 1 second to avoid rate limiting before processing begins.
3	Invoice JSON Parse1	Code	Splits multi-attachment emails: loops over all binary keys (attachment_0, attachment_1...) and creates individual items per file, preserving original email ID.
4	Filter2	Logic	Filters files by MIME type — only passes through PDF (application/pdf) or JPEG (image/jpeg) files. XML files proceed through a separate branch.
5	Loop Over Items	Logic	SplitInBatches node: processes each attachment one at a time in sequential rounds to avoid concurrency conflicts.

Phase 2: Database Logging & Counter

#	Node Name	Type	Description
6	Store Email (DB)	MySQL	Inserts email metadata (sender, subject, date, body text) into email_logs table with status='RECEIVED'.
7	Get Email ID	MySQL	Retrieves the ID of the just-inserted email log record for linking with processed files.
8	Get Settings	MySQL	Reads invoice_counter and pdf_credits from the settings table (id=1).
9	Set Counter	Set / Edit	Increments the invoice_counter by 1 in memory.
10	Update Counter (DB)	MySQL	Writes the new counter value back to the settings table.
11	Code in JavaScript1	Code	Re-attaches the binary file data from the original parser node using itemMatching(\$itemIndex) so it is available for subsequent steps.

Phase 3: File Naming & Deduplication

#	Node Name	Type	Description
12	Set File Name	Set / Edit	Extracts filename without extension from binary data (e.g. 'invoice_2025_03' from 'invoice_2025_03.pdf').
13	Search Processed PDF	MySQL	Queries processed_files table to check if this filename was already processed (deduplication check).
14	Create DB Record	MySQL	Inserts a new record in processed_files with status='PROCESSING', linking filename to email_id.
15	Code in JavaScript	Code	Re-attaches binary data again for the next stage.
16	FTP List (facturen/new/)	FTP	Lists all files in FTP facturen/new/ folder to check for naming conflicts.
17	Code in JavaScript2	Code	Generates a safe, unique filename: if 'invoice.pdf' exists, creates 'invoice_v2.pdf', etc. Carries binary data forward.
18	FTP Upload errors/	FTP	Uploads the invoice file to facturen/errors/ folder as an initial backup copy.

Phase 4: File Type Routing (Grand Router1)

The Grand Router1 switch node examines the MIME type of each file and routes it to the appropriate processing branch. XML invoices (UBL format) follow a direct parsing path, while PDF/JPEG files go through the PDF.co OCR pipeline.

#	Node Name	Type	Description
19	Grand Router1	If / Switch	Switch: Branch 1 = text/xml → XML parse path. Branch 2 = PDF/image → PDF.co OCR path + Mistral AI path.
20	Extract from File	Code	XML branch: extracts raw XML content from binary data using n8n's built-in XML extractor.
21	XML	Code	Parses XML string into a JSON object using mergeAttrs=true for UBL namespace handling.
22	converter1	Code	Transforms UBL XML JSON into a PDF.co-compatible 'body' text string, extracting supplier name, invoice ID, date, totals, and line items. Namespace-flexible parser handles ns25:, cac:, cbc: prefixes.
23	uploader	HTTP Request	PDF branch: uploads binary file to PDF.co /v1/file/upload endpoint. Returns a temporary URL for the file.
24	converting text	HTTP Request	Calls PDF.co /v1/pdf/convert/to/text to extract plain text from the uploaded PDF. Returns structured body text with preserved layout.
25	4444 (Mistral)	OpenAI	Image branch: sends JPEG invoice to Mistral Cloud API for OCR text extraction.
26	999	Set / Edit	Formats Mistral OCR output into a standard extractedText field.
27	Merge222	Merge	Combines the PDF.co text result and Mistral result into a single item for unified downstream processing.
28	Message a model4	OpenAI	GPT-4.1 reformats the raw OCR text to match the exact PDF.co body format (whitespace, alignment, structure) using a strict system prompt with a real invoice example.
29	converter	Code	Extracts body, pageCount, error, status, remainingCredits from GPT-4.1 response into a clean JSON.

Phase 5: OCR Results → DB Update & Supplier Resolution

#	Node Name	Type	Description
30	Merge1	Merge	Combines XML path result (converter1) with OCR path result (converter) by position.
31	Get ID from DB	MySQL	Retrieves the processed_files record ID for the current file (for update operations).
32	Update UBL Invoices (OCR)	MySQL	Updates processed_files: sets ocr_data = extracted text body, status = 'OCR Done'.
33	15. Update Settings (Credits)	MySQL	Updates pdf_credits in settings table with remainingCredits from PDF.co response.

34	17. Wait 20s	Wait	3-second wait to allow DB writes to complete before supplier resolution.
35	Merge	Merge	Combines OCR data with file metadata into a unified payload.
36	Edit Fields3	Set / Edit	Prepares json_text (OCR body), filename, and document ID for the A1 supplier resolver call.
37	Call A1	Execute Workflow	Calls sub-workflow 'A1 - Resolve Invoice Supplier' synchronously. Passes OCR text and document ID.
38	19. Supplier JSON (Parse)	Code	Wraps the A1 result in a supplier_info object for clean downstream referencing.
39	20. Update DB (Supplier)	MySQL	Updates processed_files: sets status='SUPPLIER_FOUND' and supplier_id from A1 result.
40	21. Wait 20s	Wait	2-second wait before calling A2 classifier.

Phase 6: Classification & Grand Routing

#	Node Name	Type	Description
41	Edit Fields	Set / Edit	Assembles the A2 payload: ocr_data, SupplierResolutionResult, document-id, supplierinfo.
42	22. Call A2 (Classifier)	Execute Workflow	Calls A2 sub-workflow synchronously. A2 returns Invoice (UBL JSON) + validation object with invoiceType and supplierMatch.
43	23. Invoice JSON (Parse)	Code	Parses A2 response: handles both string output (from AI) and pre-parsed objects (from XML path).
44	25. Wait 20s	Wait	1-second wait after A2 before routing.
45	26. Grand Router	If / Switch	The critical routing switch with 3 outputs: (1) Purchase invoice with known supplier → PO Matcher path, (2) Cost invoice with known supplier → Ledger Mapper path, (3) Unknown type or unknown supplier → DB update only.

Phase 7: Purchase Invoice Path (A3 Matcher)

#	Node Name	Type	Description
46	27. Get Open Orders	MySQL	Calls stored procedure awf_fn_supplier_open_orders(supplierId, 180) on Prod DB to get all open POs for this supplier in the last 180 days.
47	28. Parse Open Orders	Code	Parses the JSON string returned by the stored procedure. Handles empty results gracefully.
48	29. Orders Found?	If / Switch	If open orders exist (items.length >= 1) → calls A3 Matcher. If no orders → calls A3 with empty orders (for cost matching fallback).
49	Edit Fields4/10	Set / Edit	Prepares invoice, validation, and purchase_orders for A3 input.
50	30. Call A3 (Matcher)	Execute Workflow	Calls A3 Purchase Order Matcher sub-workflow. A3 returns matched/unmatched result with orderMatch.
51	Clean & Parse JSON4	Code	Cleans A3 output: strips markdown code blocks, parses JSON.
52	32. Output JSON (A3)2	Code	Passes clean A3 output downstream.
53	33. DB Update (Matched)2	MySQL	Updates aws_invoice_json: sets validation_json, status (MATCHED or NEW), ubl_json, updated_at.
54	Edit Fields19	Set / Edit	Serialises invoice and validation for file conversion.
55	19. Parse6 → Convert5	Code	Parses strings, converts to JSON file (filename.json).
56	40. FTP Move (Success)	FTP	Uploads the JSON output file to FTP facturen/new/ folder.
57	FTP Rename	FTP	Renames the file from facturen/errors/ to facturen/new/ (moves it from backup to processed).
58	Loop Over Items	Logic	Loops back to process the next attachment in the batch.

Phase 8: Cost Invoice Path (A4 Ledger)

#	Node Name	Type	Description
59	Edit Fields5	Set / Edit	Prepares invoice and validation (as JSON strings) for A4 Ledger Mapper input.
60	35. Call A4 (Ledger)	Execute Workflow	Calls A4 Ledger Mapper sub-workflow synchronously. Returns enriched invoice with GL account mappings.
61	Clean & Parse JSON1	Code	Cleans and parses A4 response JSON.
62	36. Invoice JSON (A4)	Code	Normalises A4 output structure.
63	Edit Fields8	Set / Edit	Prepares invoice and validation strings for DB update.
64	37. DB Update (Ledger)	MySQL	Updates aws_invoice_json with ledger-mapped validation_json and ubl_json. Sets status MATCHED or REVIEW.
65	Convert + FTP	Convert / FTP	Converts result to JSON file and uploads to FTP facturen/new/.

Main Flow — Simplified Flowchart

The following chart shows the high-level sequence of the Main Flow:



Figure 1a: Main Flow — Phase 1: Email Ingestion to OCR (continued below)

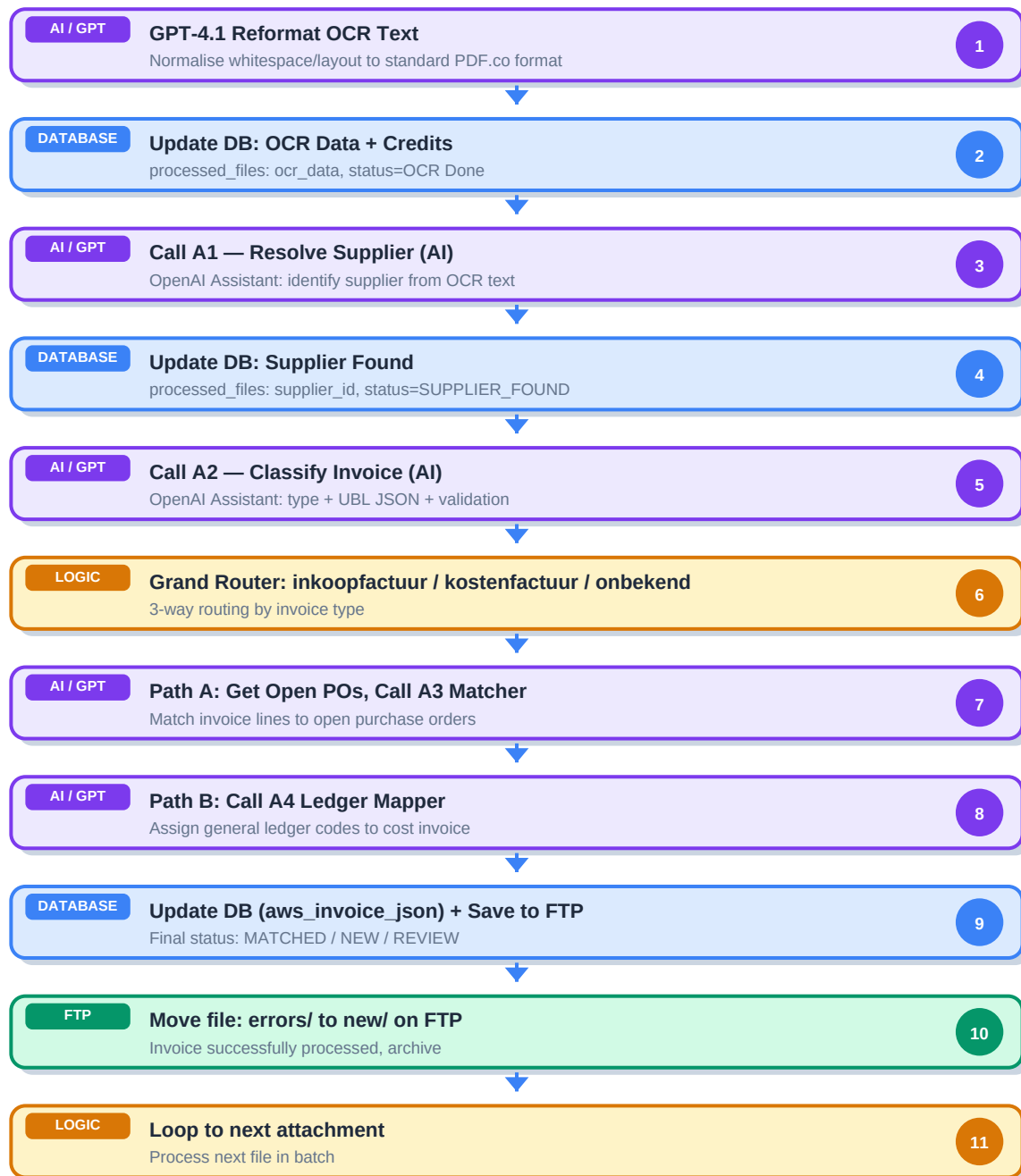


Figure 1b: Main Flow — Phase 2: AI Processing to Final Storage

4. A1 — Resolve Invoice Supplier

Workflow Purpose

The A1 sub-workflow receives the OCR-extracted text of an invoice and uses an OpenAI Assistant (equipped with a Vector Store of supplier knowledge chunks) to identify the correct supplier (creditor) from the company's supplier database. It returns a structured JSON response with a supplierId, confidence score, and match evidence. This is a critical step — if the supplier cannot be resolved, the invoice is flagged and no further processing occurs.

Trigger Execute Workflow Trigger — called by Main Flow node '37. Call A1'

Input Fields json_text (OCR body string), id (document ID from processed_files)

Output supplierId (int), confidence (float 0-1), evidence (array), Supplier.matched (bool), Supplier.supplierId, Supplier.fuzzyMatchScore

AI Assistant Details

#	Node Name	Type	Description
1	Execute Workflow Trigger	Logic	Entry point when Main Flow calls this sub-workflow. Receives: json_text (OCR content) and document id.
2	Supplier Resolver (AI)1	OpenAI	OpenAI Assistant 'Copy of A1 Resolve Invoice Supplier N8N' (asst_mzuHs96x8HxGf8qwp3x9sYsX). Uses Vector Store with supplier knowledge chunks. Prompt: 'Find the supplier for this invoice text: [OCR text]'. Returns JSON with supplierId, confidence, Supplier object.
3	Parse Response1	Code	Extracts the first valid JSON object from AI output using brace-counting algorithm. Robust against AI adding extra text or comments.
4	Did AI Work?	If / Switch	Checks if \$json.error is empty (meaning AI succeeded). YES → continue. NO → return AI Fail Response.
5	Update rows in table	MySQL	On success: updates processed_files table — sets supplier_id, confidence_score, status='SUPPLIER_FOUND'. Matched to document ID from trigger.
6	Parse Response	Code	Final normalisation: pulls aiData from Parse Response1 for the Switch node.
7	Switch (Matched?)	If / Switch	Checks Supplier.matched boolean. Matched=true → Success Response. Matched=false → Fallback Response.
8	Success Response	Logic	Responds to Main Flow with full supplier JSON (200 OK).
9	Fallback Response (Fail)	Logic	Returns supplierId=0, matched=false (200 OK) — Main Flow handles gracefully.
10	AI Fail Response	Logic	Returns supplierId=0, error='AI Failed' (200 OK) — prevents Main Flow crash.

A1 — Flowchart

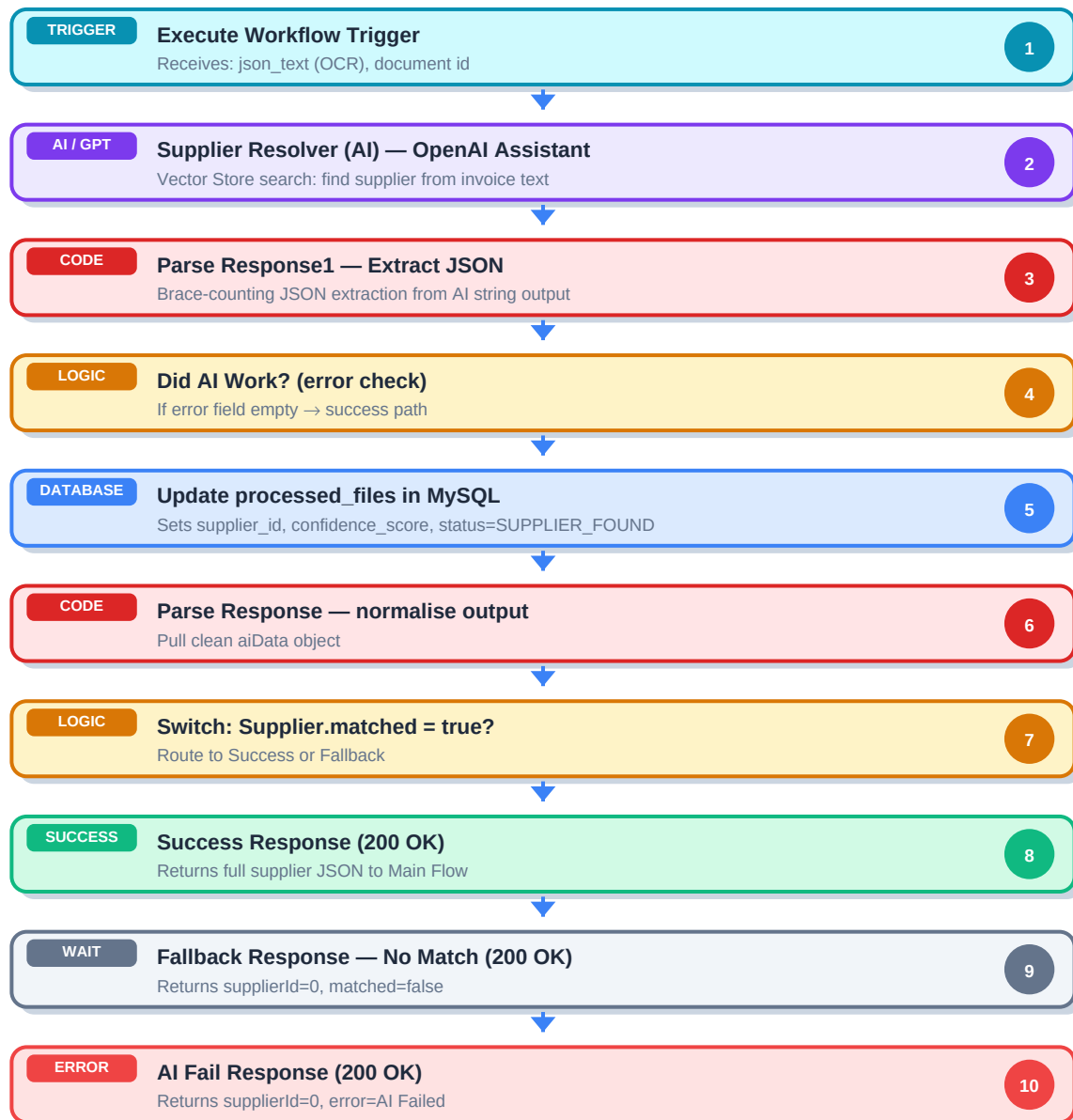


Figure 2: A1 — Supplier Resolution Sub-Workflow

5. A2 — Invoice Classifier & Transformer

Workflow Purpose

The A2 sub-workflow is the intelligence core of the system. It receives the OCR text and the supplier resolution result, then uses an OpenAI Assistant to: (1) classify the invoice as 'inkoopfactuur' (purchase invoice) or 'kostenfactuur' (cost invoice) or 'onbekend' (unknown), (2) transform the raw OCR text into a structured UBL-format JSON Invoice object, and (3) produce a validation object with supplierMatch, invoiceType, status, and metadata. The result is stored in the aws_invoice_json table for downstream processing.

Trigger	Execute Workflow Trigger — called by Main Flow node '22. Call A2 (Classifier)'
Input Fields	ocr_data, SupplierResolutionResult, document-id, supplierinfo
Output	Invoice (UBL JSON object), validation {invoiceType, supplierMatch, status, metadata}

Node-by-Node Description

#	Node Name	Type	Description
1	Execute Workflow Trigger	Logic	Entry point. Receives ocr_data, supplierinfo, document-id from Main Flow.
2	Invoice Classifier (AI)1	OpenAI	OpenAI Assistant 'Copy of A2 Invoice Classifier & Transformer N8N' (asst_PogHknCOyBE2B9Sw1LfvhfDs). Prompt sends ocr_data + SupplierResolutionResult. Returns structured JSON: {Invoice: {...UBL format...}, validation: {invoiceType, supplierMatch, status, metadata}}.
3	Did AI Work?1	If / Switch	Error check: if \$json.error is empty → AI succeeded. Routes to Parse JSON. If error exists → AI Error Response.
4	Parse JSON Node Name	Code	Simple JSON.parse of \$json.output string. Returns clean Invoice + validation objects.
5	Did AI Work?	If / Switch	Secondary check: verifies validation.status != 'not_invoice' AND supplierMatch.supplierId exists. Both conditions must pass.
6	SQL Lookup Node	MySQL	Looks up artikelgroep (article group) table to find the matching category ID for the invoice type. Uses LIKE query on omschrijving field.
7	SQL Lookup Node1	MySQL	Updates aws_invoice_json table: sets supplier_id, invoice_type ('cost'/'purchase'/'none'), issue_date, total_payable, ubl_json, validation_json, status, currency — matched on document_id.
8	Parse Response	Code	Assembles final output: {Invoice, validation, processing_metadata: {document_id, category_id, supplier_id, status}}.
9	Switch (Matched?)	If / Switch	Checks validation.supplierMatch. Matched → Success Response. Not matched → Fallback Response.
10	Success Response	Logic	Returns full Invoice+validation JSON to Main Flow (200 OK).
11	Fallback Response	Logic	Returns generic no-match response (200 OK).
12	AI Error Response	Logic	Returns error JSON (500) if AI failed or output was not parseable.

Invoice Type Classification Logic

The AI Assistant classifies each invoice into one of three categories based on the OCR content and supplier information. The classification drives all downstream routing in the Main Flow's Grand Router node.

Invoice Type (Dutch)	English Meaning	Downstream Action	DB Status
inkoopfactuur	Purchase Invoice	Call A3 PO Matcher — match to open purchase orders	MATCHED or NEW
kostenfactuur	Cost/Expense Invoice	Call A4 Ledger Mapper — assign GL accounts	MATCHED or REVIEW
onbekend	Unknown	No further AI processing — flagged for manual review	REVIEWED

A2 — Flowchart



Figure 3: A2 — Invoice Classifier & Transformer Sub-Workflow

6. A3 — Purchase Order Matcher

Workflow Purpose

The A3 sub-workflow handles purchase invoices (inkoopfactuur). It receives the UBL-format Invoice JSON, the validation result from A2, and the list of open purchase orders (from the ERP database). An OpenAI Assistant then attempts to match each invoice line item to a corresponding purchase order line. The result includes an orderMatch object indicating whether a match was found, the matched order ID, and the confidence level.

Trigger	Execute Workflow Trigger — called by Main Flow nodes '30. Call A3 (Matcher)' and '30. Call A3 (Matcher)2'
Input Fields	invoice (UBL JSON string), validation (JSON string), purchase_orders (JSON string from ERP stored procedure)
Output	Invoice + validation with orderMatch: {matched (bool), orderId, orderLines, confidence}

Node-by-Node Description

#	Node Name	Type	Description
1	Execute Workflow Trigger	Logic	Entry point. Receives invoice, validation, and purchase_orders as JSON strings from Main Flow.
2	Parse & Count Orders	Code	Parses all three input strings: JSON.parse(input.invoice), JSON.parse(input.validation), JSON.parse(input.purchase_orders). Converts strings to real JSON objects for the AI.
3	Purchase Order Matcher (A3)	OpenAI	OpenAI Assistant 'Copy of A3 Purchase Order Matcher N8N' (asst_o7CcGHLadsypmucRXatWXDT1). Prompt sends Invoice, validation, and purchase_orders as a combined JSON. Returns matched result with orderMatch details.
4	Code in JavaScript1	Code	Parses AI output string: JSON.parse(\$json.output). Handles parsing errors gracefully.
5	Final Success Response	Logic	Returns the complete matched/unmatched invoice+validation JSON to Main Flow (200 OK). Main Flow then updates the database.

Purchase Order Data Source

Open purchase orders are fetched by the Main Flow before calling A3, using a MySQL stored procedure on the Production database:

Stored Procedure	awf_fn_supplier_open_orders(supplierId, 180) Returns all open POs for the supplier created in the last 180 days as a JSON string. Uses GROUP_CONCAT with 64MB limit to handle large PO datasets.
------------------	--

A3 — Flowchart

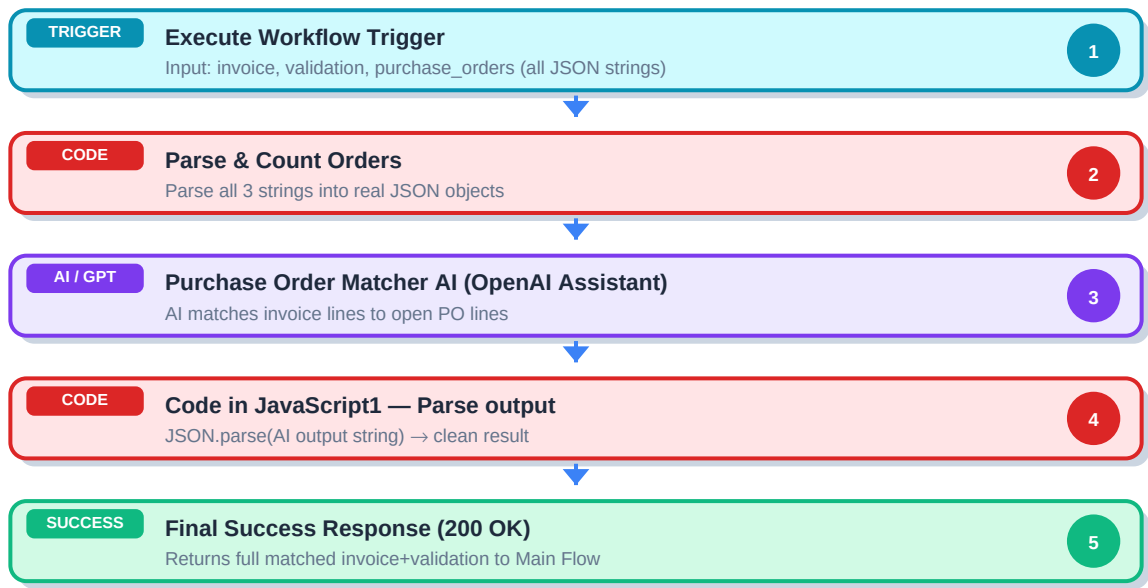


Figure 4: A3 — Purchase Order Matcher Sub-Workflow

Matching Outcomes

Outcome	DB Status	Description
orderMatch.matched = true	MATCHED	AI found a matching PO with high confidence. Invoice is automatically approved for processing.
orderMatch.matched = false	NEW	No matching PO found. Invoice is flagged as NEW for manual review by accounts payable.

7. A4 — Ledger Mapper

Workflow Purpose

The A4 sub-workflow handles cost invoices (kostenfactuur). It receives the Invoice and validation JSON from A2 and uses an OpenAI Assistant (Ledger Matcher) to assign the correct general ledger (grootboekrekening) account codes to each invoice line item. The Assistant has been trained with the company's chart of accounts and historical booking patterns. The result is returned to the Main Flow for database storage and FTP archiving.

Trigger	Execute Workflow Trigger — called by Main Flow node '35. Call A4 (Ledger)'
Input Fields	invoice (UBL JSON string), validation (JSON string) — from A2 output
Output	Invoice + validation enriched with ledger account codes (grootboekrekening) per line item

Node-by-Node Description

#	Node Name	Type	Description
1	Execute Workflow Trigger	Logic	Entry point. Receives invoice and validation as JSON strings from Main Flow.
2	Parse Input JSON	Code	Robust JSON parser using safeParseFirst() function: handles concatenated JSON objects, trailing garbage, and malformed strings. Parses both invoice and validation to proper objects.
3	Ledger Matcher	OpenAI	OpenAI Assistant 'Copy of A4 Ledger Mapper N8N' (asst_TqqIQihpuUeeumOzVeLeZkSj). Sends {Invoice: ..., validation: ...} to AI. Returns full Invoice JSON enriched with ledger codes for each invoice line.
4	Parse Input JSON1	Code	Advanced parser: strips markdown code blocks (``json), removes AI comments (//) from output, finds first { and last } to extract clean JSON.
5	Did AI Work?	If / Switch	Checks if validation.status == 'ok' OR 'ok_with_warning'. If yes → Success Response. If no → AI Error Response.
6	Success Response	Logic	Returns ledger-mapped invoice JSON to Main Flow (200 OK).
7	AI Error Response	Logic	Returns {error: 'AI Ledger Mapping Failed'} with 500 status code if AI fails.

A4 — Flowchart

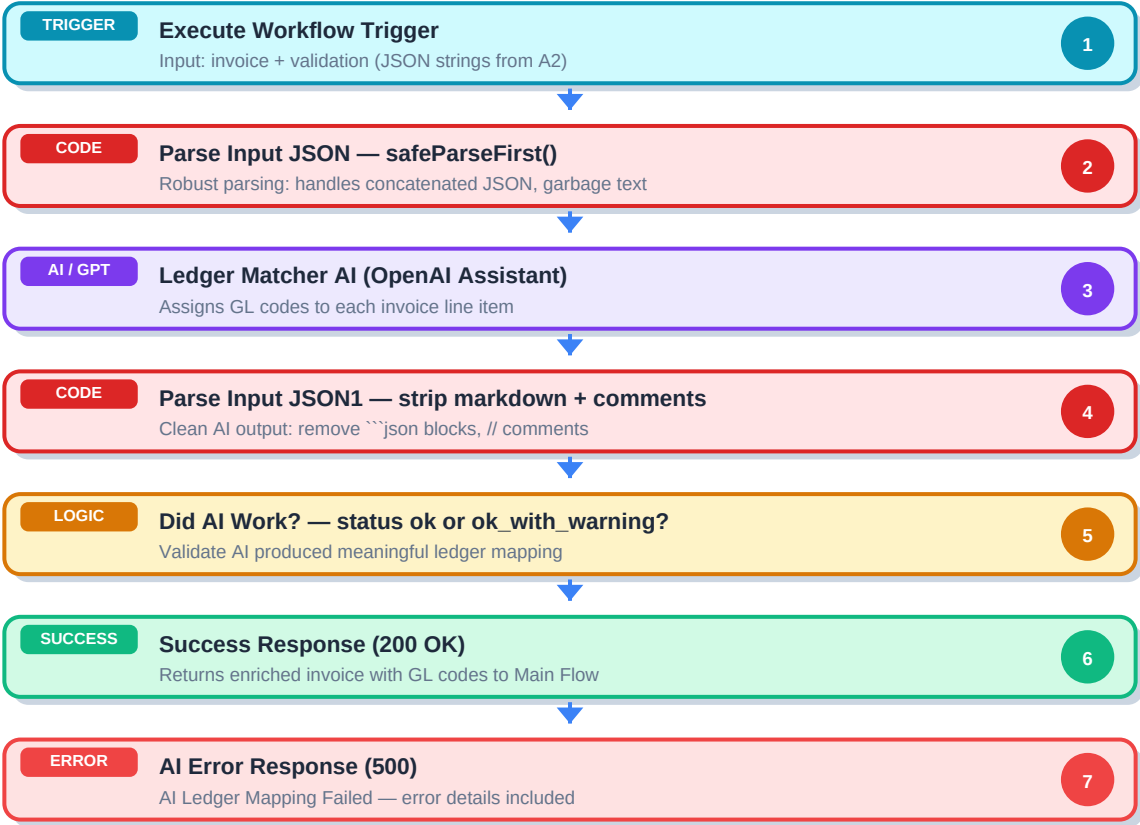


Figure 5: A4 — Ledger Mapper Sub-Workflow

8. Vector Store Updater — Supplier Knowledge Base

Workflow Purpose

The Vector Store Updater maintains the OpenAI Vector Store that powers the A1 Supplier Resolver AI Assistant. Each supplier in the system has a corresponding Markdown 'chunk' file in the vector store containing their company profile, known invoice formats, and typical ledger codes. When a supplier record is updated or created in the database, this workflow regenerates the chunk, enriches it with GPT-4.1, and uploads it to the OpenAI Vector Store (vs_6826ef5e64a0819182f331d03fe628e8). This ensures the AI always has up-to-date knowledge about each supplier.

Trigger	Manual Trigger (can be scheduled). Processes all suppliers marked vs_bUpdate=1 or vs_bDelete=1.
----------------	---

Vector Store ID	vs_6826ef5e64a0819182f331d03fe628e8 — OpenAI Assistants API v2
------------------------	--

Three Processing Paths

The workflow handles three scenarios: (1) Deleting obsolete supplier chunks, (2) Updating existing supplier chunks (remove old + create new), and (3) Creating new supplier chunks from scratch.

Path 1: Delete Suppliers (vs_bDelete=1)

#	Node Name	Type	Description
1	Suppliers to be deleted1	MySQL	SELECT rc_ID, vs_file_id WHERE vs_bDelete=1 from awf_supplier_chunk (Prod DB).
2	Route Existing/Deleted2	If / Switch	If vs_file_id is not empty → file exists in OpenAI → delete it first.
3	Remove from Vector Store2	HTTP Request	DELETE https://api.openai.com/v1/vector_stores/{vs_id}/files/{file_id}
4	Remove from OpenAI Storage2	HTTP Request	DELETE https://api.openai.com/v1/files/{vs_file_id}
5	Delete DB Record	MySQL	DELETE FROM awf_supplier_chunk WHERE rc_ID = ...

Path 2: Update Suppliers (vs_bUpdate=1, existing file)

#	Node Name	Type	Description
1	Suppliers to be updated1	MySQL	SELECT rc_ID, vs_file_id, supplierID, vs_filename WHERE vs_bUpdate=1 (Prod DB).
2	Route Existing/Deleted1	If / Switch	If vs_file_id != 'null' → existing file in vector store → delete old version first.
3	Remove from Vector Store1	HTTP Request	Removes old file from vector store.
4	Remove from OpenAI Storage1	HTTP Request	Removes old file from OpenAI file storage.

5	Create Chunk Part2	MySQL	Calls stored procedure awf_fn_generate_supplier_md(supplierID) — generates base Markdown profile with company info, addresses, bank details.
6	detect missing Ledger	MySQL	Finds historical GL codes used for this supplier (from kostenfactuur table) — enriches ledger section.
7	Create Prompt2	MySQL	Calls awf_fn_generate_supplier_md_enrichment_prompt(supplierID) — generates AI enrichment prompt.
8	Loop Over Items	Logic	Processes in batches of 10 suppliers to avoid API rate limits.
9	OpenAI (ChatGPT)2	OpenAI	GPT-4.1: merges base chunk + ledger codes into one unified Markdown document. Removes duplicate entries and old invoice lines (before Aug 2024). Returns clean Markdown.
10	Make Binary2	Code	Combines Chunk_Part1 + AI enrichment into final Markdown. Encodes as base64 binary file.
11	Add To Storage2	OpenAI	Uploads Markdown file to OpenAI file storage (purpose=assistants).
12	Add to Vector Store3	HTTP Request	POST to vector_stores/{vs_id}/files with new file_id.
13	Data store (Update)2	MySQL	Updates awf_supplier_chunk: sets vs_file_id = new file ID, vs_bUpdate = 0.

Vector Store Updater — Flowchart (Delete Path)

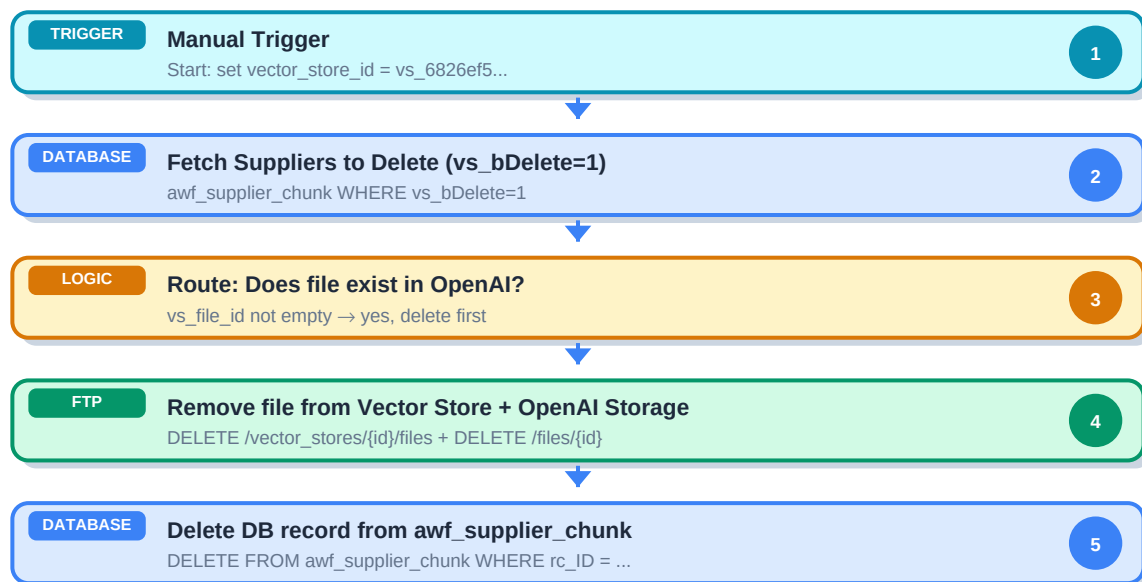


Figure 6a: Vector Store Updater — Delete Path

Vector Store Updater — Flowchart (Update/Create Path)

*Figure 6b: Vector Store Updater — Update/Create Path*

9. Database Schema & Tables

MySQL Tables Used

Table: email_logs

Column	Data Type	Description
id	INT AUTO_INCREMENT PK	Unique email record ID
sender	VARCHAR(255)	Email sender address
subject	VARCHAR(255)	Email subject line
date	DATETIME	Email received timestamp (NOW())
body	TEXT	Email plain text body
status	VARCHAR(50)	Processing status (RECEIVED, etc.)

Table: processed_files

Column	Data Type	Description
id	INT AUTO_INCREMENT PK	Unique file record ID
filename	VARCHAR(255)	Invoice filename without extension
status	VARCHAR(50)	Processing stage (PROCESSING, OCR Done, SUPPLIER_FOUND, etc.)
email_id	INT FK → email_logs.id	Linked email record
ocr_data	LONGTEXT	Extracted OCR text body
supplier_id	INT	Resolved supplier ID (from A1)
confidence_score	FLOAT	AI confidence 0.0-1.0 (from A1)

Table: aws_invoice_json

Column	Data Type	Description
id	INT AUTO_INCREMENT PK	Unique invoice JSON record ID
document_id	INT FK → processed_files.id	Linked processed file
ubl_json	LONGTEXT	Full UBL-format Invoice JSON (from A2)
validation_json	LONGTEXT	Validation object JSON (from A2, A3, A4)
supplier_id	INT	Supplier ID
invoice_type	VARCHAR(20)	cost / purchase / none
issue_date	DATE	Invoice date
total_payable	DECIMAL(10,2)	Total invoice amount
currency	VARCHAR(10)	Currency code (EUR, etc.)
status	VARCHAR(20)	MATCHED / NEW / REVIEW / REJECTED
updated_at	DATETIME	Last update timestamp

Table: settings

Column	Data Type	Description
id	INT PK (always 1)	Single settings row
invoice_counter	INT	Auto-incrementing invoice counter
pdf_credits	INT	Remaining PDF.co API credits

Table: awf_supplier_chunk

Column	Data Type	Description
rc_ID	INT PK	Chunk record ID
supplierID	INT	Linked supplier ID in ERP
vs_file_id	VARCHAR(100)	OpenAI file ID in vector store
vs_filename	VARCHAR(255)	Markdown filename (e.g. leverancier-42.md)
vs_bUpdate	TINYINT(1)	Flag: 1 = needs rebuild, 0 = current
vs_bDelete	TINYINT(1)	Flag: 1 = mark for deletion

10. Error Handling & Fallback Strategy

System-Wide Error Philosophy

The CocktailMachines automation system is designed for resilience. Every AI call and database operation has a fallback response to prevent the entire pipeline from crashing on a single invoice failure. Errors are logged, files are kept in FTP facturen/errors/ until successfully processed, and the Main Flow continues to the next attachment even if one fails.

Error Handling by Layer

Layer	Error Scenario	Handling Strategy
HTTP Nodes (PDF.co, OpenAPI)	API timeout or server error	retryOnFail=true, maxTries=3, waitBetweenTries=5000ms
A1 Supplier Resolver	AI fails to parse supplier	Returns supplierId=0, matched=false — Main Flow continues
A1 Supplier Resolver	No supplier match found	Fallback Response (200): supplierId=0, Main Flow flags as unknown
A2 Classifier	AI output not parseable	AI Error Response (500) — logged in DB
A2 Classifier	not_invoice status	Invoice rejected — status=REJECTED in DB
A3 PO Matcher	No open purchase orders	AI called with empty orders list — may still return matched=false
A3 PO Matcher	No PO match found	DB status=NEW — flagged for manual accounts payable review
A4 Ledger Mapper	AI fails ledger mapping	Error response — status=REVIEW for manual processing
FTP Operations	File already exists	Versioning: filename_v2.pdf, filename_v3.pdf etc.
MySQL Queries	Node error	onError=continueRegularOutput — flow continues
Vector Store Update	OpenAI API error	onError=continueRegularOutput on all VS update nodes
OCR (PDF.co)	API credits exhausted	onError=continueRegularOutput — Mistral fallback available

Invoice Status State Machine

Every invoice moves through a defined set of statuses in the processed_files and aws_invoice_json tables. The status reflects the current processing stage and enables monitoring and manual intervention when needed.

Status	Table	Meaning
PROCESSING	processed_files	File received and DB record created — processing started
OCR Done	processed_files	Text extracted from PDF/XML — ready for AI analysis
SUPPLIER_FOUND	processed_files / aws_invoice_json	A1 resolved the supplier — supplierId set
MATCHED	aws_invoice_json	A2/A3/A4 found a match — fully automated processing complete
NEW	aws_invoice_json	No PO match found — requires manual review by AP team
REVIEW	aws_invoice_json	A4 ledger mapping needs verification — queued for human review

REJECTED	aws_invoice_json	Not an invoice (not_invoice) or unknown supplier — excluded
----------	------------------	---

Document prepared for CocktailMachines B.V. — Internal Technical Documentation — Confidential
n8n Invoice Automation System | v1.0 | March 2025 | All workflows deployed on shared n8n instance